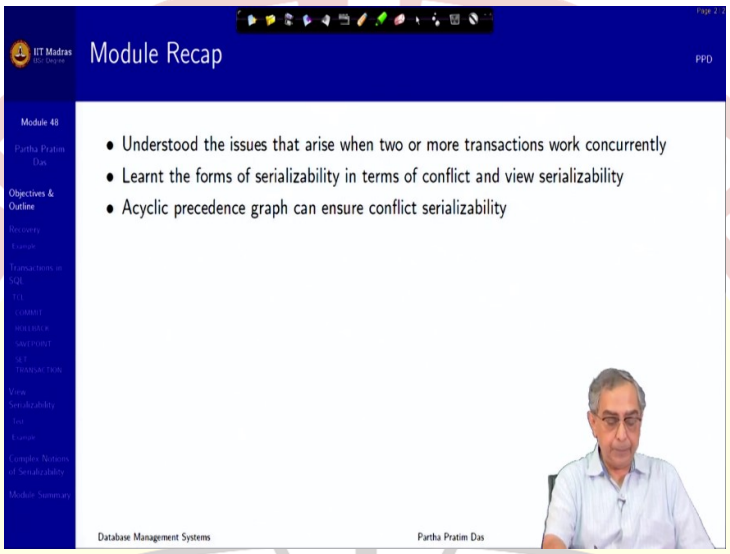
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Transactions/3: Recoverability

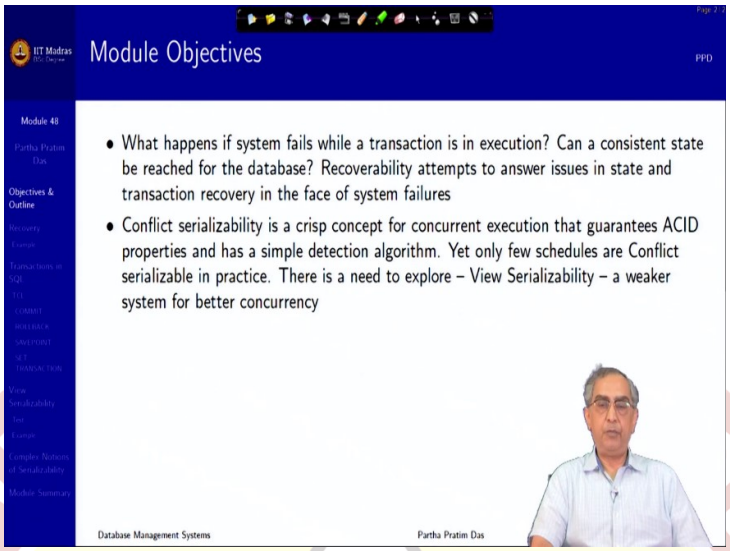
Welcome to Module 48 of Database Management Systems course, in the IIT Madras Online BSc program.

(Refer Slide Time: 00:36)



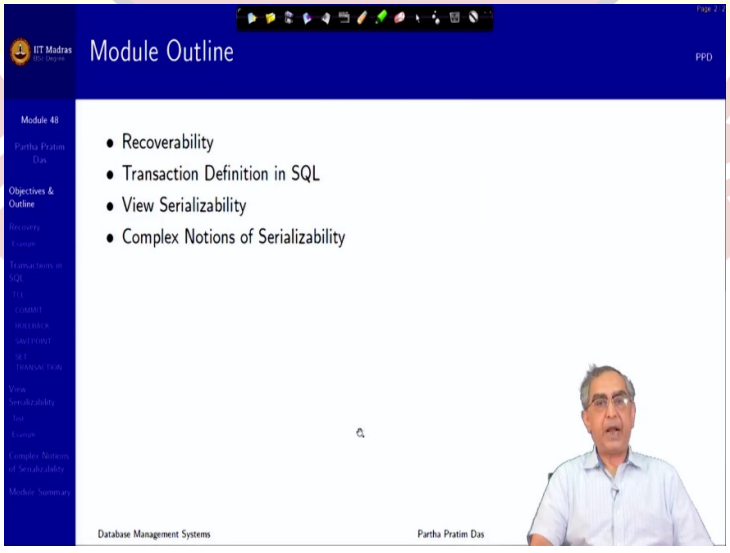
We have been discussing about transactions, and we have taken a note of the issues that arise when two or more transactions work concurrently, on the same set of data items, and learned the forms of serializability conflict and view serializability that may be used to serialize concurrent transactions. Learnt about acyclic precedence graph that can ensure conflict serializability.

(Refer Slide Time: 01:05)



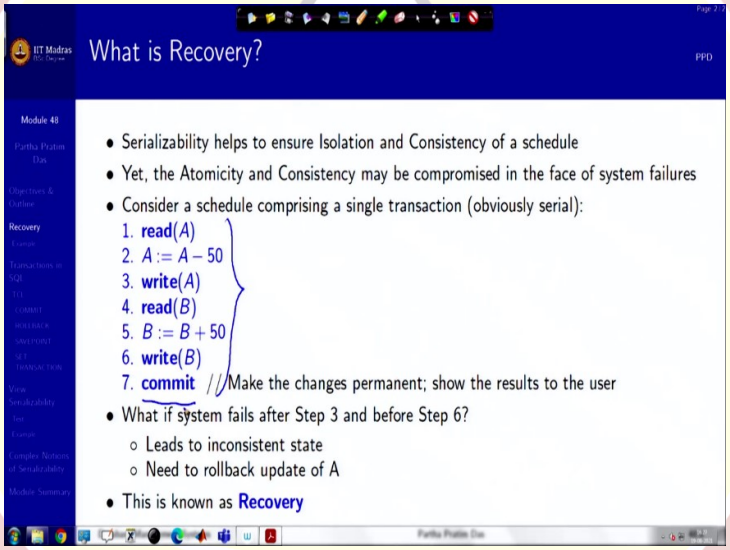
Going forward in this module, we will start by taking a look at the issues that will happen when a certain transaction fails because of system failure. There may be power outage, there may be hardware crash, software crash, any of that. Then can a consistent state be reached for the database? This is called the recoverability issue. So, we will talk about that. And we will continue on the serializability, as well, talking about view serializability which is a weaker form of serializability, which guarantees serializability of more schedules than what is guaranteed by conflict serializability.

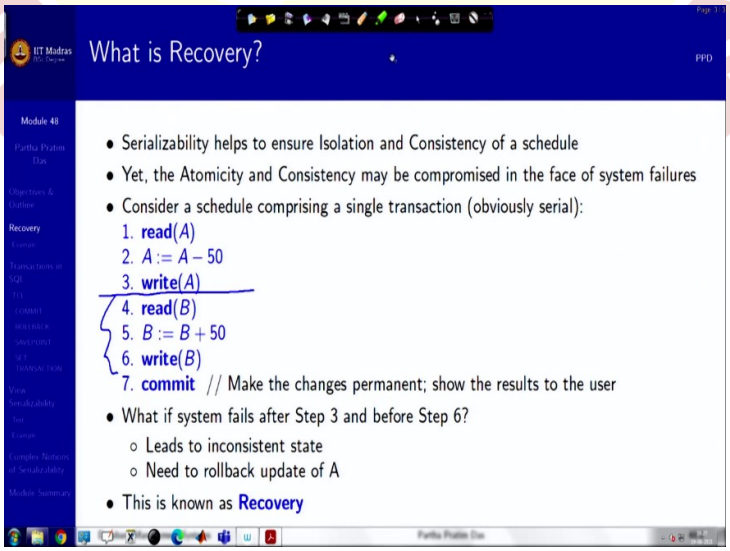
(Refer Slide Time: 01:56)



These are the primary topics we deal with.

(Refer Slide Time: 02:04)

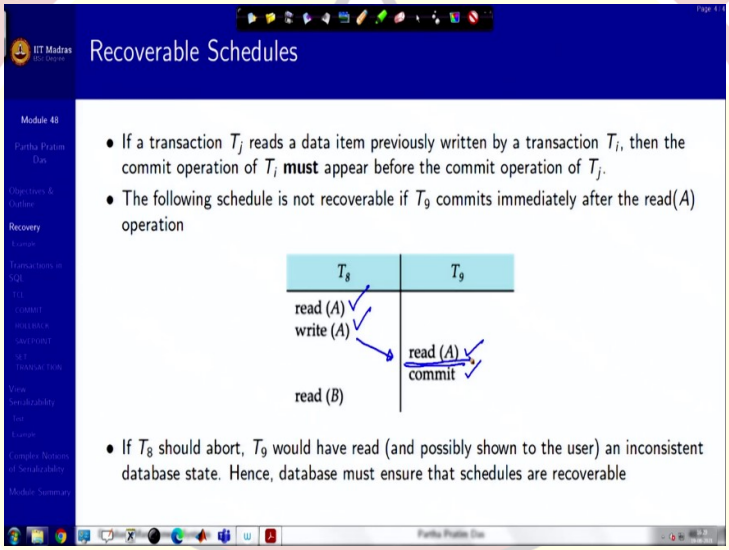




So, we start with recovery. So, serializability has helped us to ensure the kind of isolation and consistency of a schedule, but still, the atomicity and consistency may be compromised if there is a system failure. So, there is a serial schedule which is transferring 50 rupees from account A to account B, and after the commit naturally the results will be seen visible to the user in the database.

But what if the system goes wrong? System fails after A has been written. It fails somewhere before the commit has happened. What will happen between 4, 5, 6? This will certainly lead to an inconsistent state because A has already been changed. So, we need to roll back the changes in A, so that we can reach the earlier consistent state.

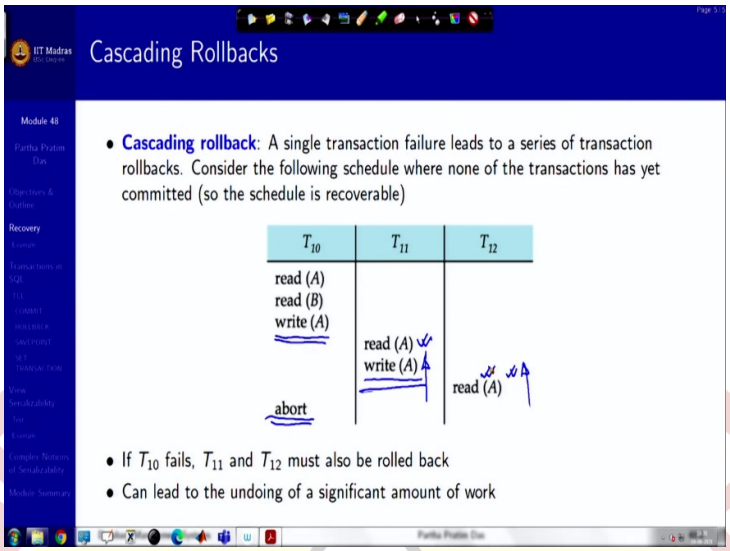
(Refer Slide Time: 03:19)



This is known as the recovery problem. So, if a transaction Tj reads a data item, which was previously written by a transaction Ti, then the recoverability kind of would be guaranteed if the commit of Ti, which wrote must happen before the commit operation of Tj. So, that is kind of the ground rule we will see.

For example, this particular this schedule is not recoverable. It reads A writes A, and then, T9 reads the A written by T8 and does a commit before the committing T8 which actually wrote A. Now, if T8 will abort T8 will fail then T9 possibly will show a inconsistent state to the user because T9 is using a value of A, which is not, maybe, which may not be correct. So, this recoverability is a is an important factor.

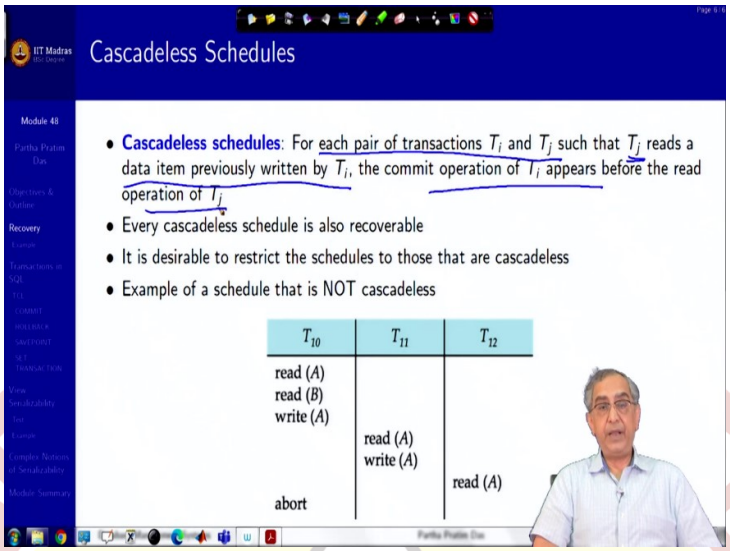
(Refer Slide Time: 04:43)



So, for this, whenever there is an abort, we need to do a roll back to the beginning of the transaction, undo everything that was done earlier. So, let us consider the following schedule where nothing has been committed yet. So, what happens? So, if there is an abort in T10, T10 will have to be rolled back if T10 fails it will have to be rolled back. If it rolls back it will roll back this write A, which will mean that the read A that T11 did gets invalidated as well, because that A is no more available T10 is undoing it.

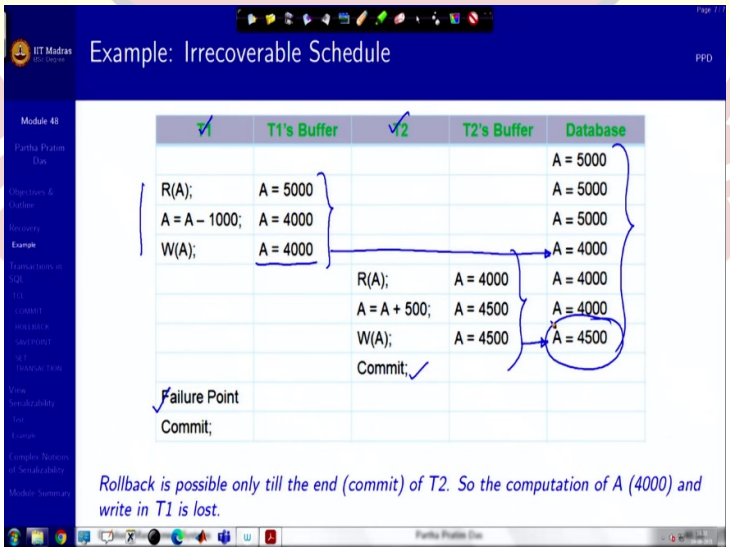
So, T11 must also be rolled back. Same thing can be said about T12, so that will also have to be rolled back because T12 is using what T11 wrote. And once T11 rolls back that is undone, this value also becomes invalid. So, you can see that is a chain reaction that would start when T10 fails. So, this is called cascading roll back. One roll back causing another roll back causing another roll back and so on.

(Refer Slide Time: 06:10)



So, this could be quite a long process. So, we would want preferred those kinds of schedules which is cascadeless, but is also recoverable. So, if we can guarantee that each pair of transaction Ti, Tj such that Tj reads a data item previously written by Ti the commit operation of Ti appears before the read of this Tj. If that is done, then it will guarantee a cascadeless recovery. So, it is it is desirable to restrict schedules to those, which are cascadeless. Cascade I mean, if a schedule is recoverable with cascading also we can, but it has a performance penalty.

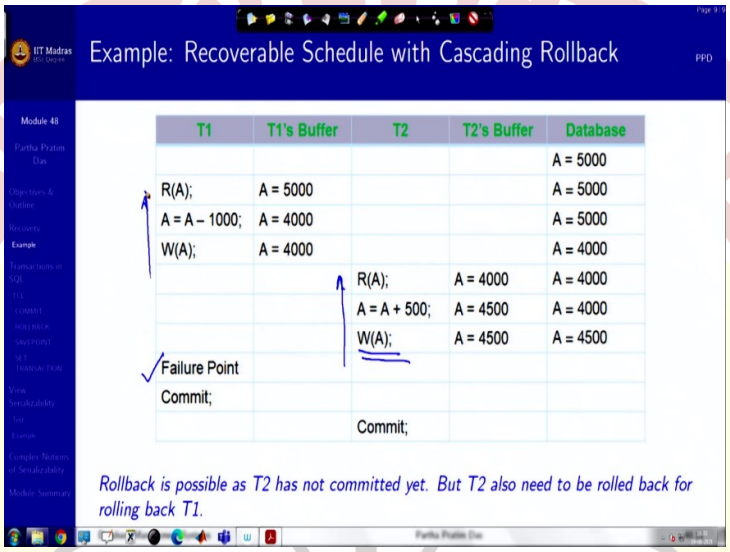
(Refer Slide Time: 07:08)



So, let us look at a few schedules. There are two schedules, T1 and T2 and these are the things being done. This is what is there in the buffer because commit has not happened.

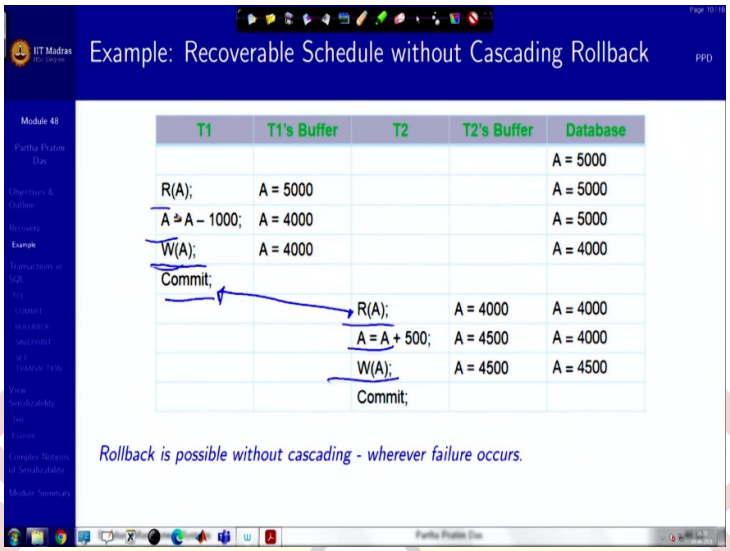
Similarly, this is what in the database and this is in the buffer of T2. So, when you do, the buffer was changing when you do a write, this changes in the database, when you do a write, this again changes in the database and then you have committed. So, you said this is done. And then you have, then T1 has failed. Now, as T1 has failed, it needs to rollback. But now, in the database, the value has already been changed and made permanent by T2, so the rollback is not possible. So, this is not a not a recoverable schedule.

(Refer Slide Time: 08:02)



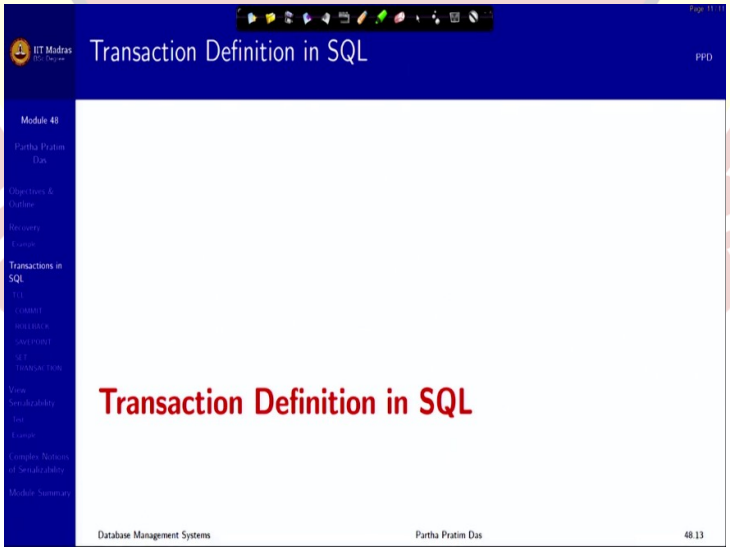
If instead, we instead of committing write after earlier T2 was committing here. Instead of doing that, if it commits after the commit of T1 then the schedule is recoverable. Because if it fails at this point before T1 has committed then this has not yet been committed. So, this can be rolled back, and then this can be rolled back. So, it leads to a cascading roll back, but rollback is possible.

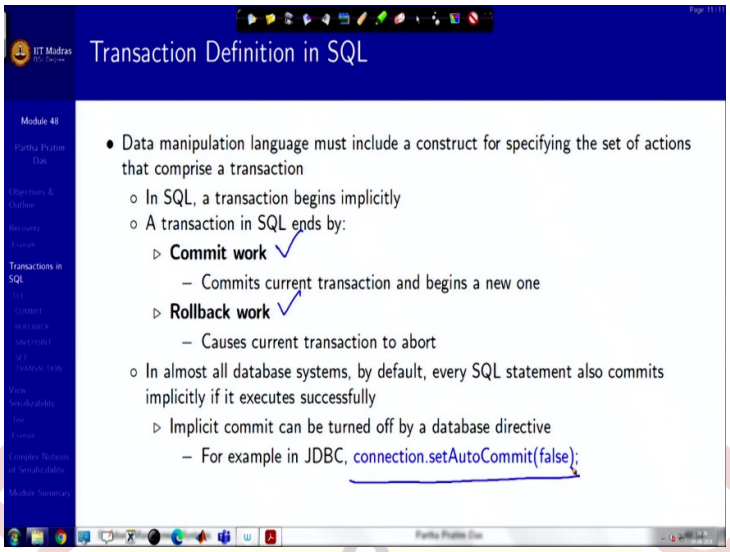
(Refer Slide Time: 08:43)



Let us look at a third, where, as soon as W(A) is done, writing is done R(A) is committed. So, what you are guaranteeing is, that before the read of T2 of A the value it is reading of A that has been committed. So now, it does not matter where it fails. If it fails somewhere here, only T2 will be rolled back, otherwise, it can fail somewhere here where T2 has not happened. So, this is a rollback is possible without cascading. This eventually becomes a kind of serial schedule, but we will see the same issues in the concurrent schedules as well.

(Refer Slide Time: 09:29)

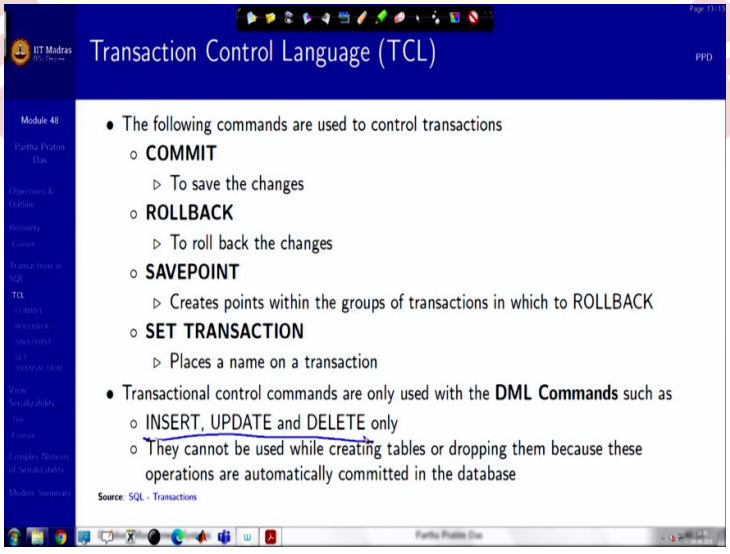




We will come back to this. These concepts are important with serializability, so we will keep coming back to them. Before we move further let us see how to write transactions in SQL. So, in SQL you can actually write a transaction. So, the two primary commands for transactions are commit, which a transaction ends with or rollback, which will cause the current transaction to abort. These are the, and take it back to the beginning of the transaction or some other point.

Now, in almost all database systems, every SQL statement commits implicitly, if it executes successfully, otherwise it rolls back. If it is not, if you do not want that, then you can forcibly tell the database not to do implicit commit. Like in JDBC, you can say this to put off the implicit commit.

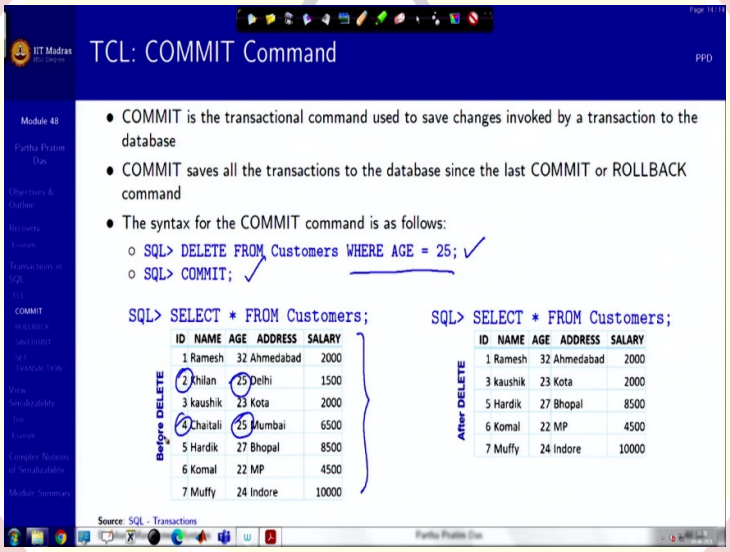
(Refer Slide Time: 10:30)

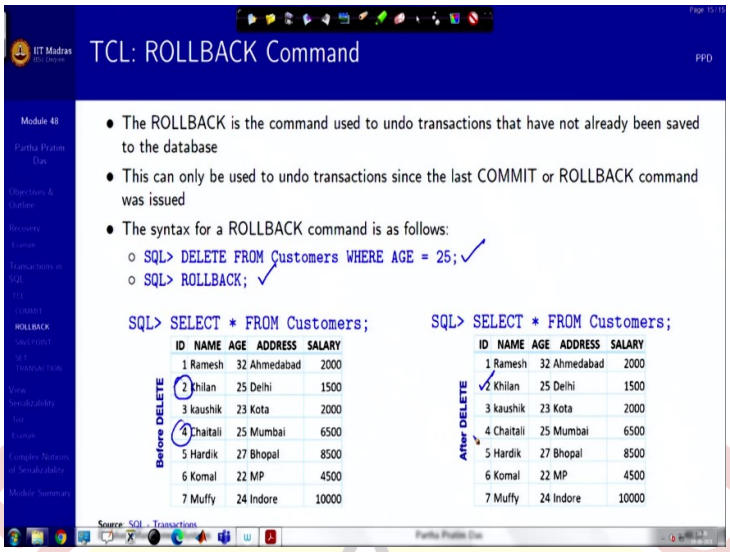


So, in there are four different commands, primary commands in TCL, called Transaction Control Language. So, there is a part of SQL, which relates to the transactions. One is COMMIT to save the changes, one is ROLLBACK changes, SAVEPOINT is kind of keeping a marker that if then a group of transaction, you want to roll back to a certain point not necessarily to the beginning, then you can set a SAVEPOINT, and SET TRANSACTION can put a name on a transaction.

Remember, that this commands are available with insert, delete, update only. Other data manipulation commands like create table or dropping tables and so on, they cannot be managed by the TCL commands, they are managed, they are commit and rollback and managed by the database itself.

(Refer Slide Time: 11:22)

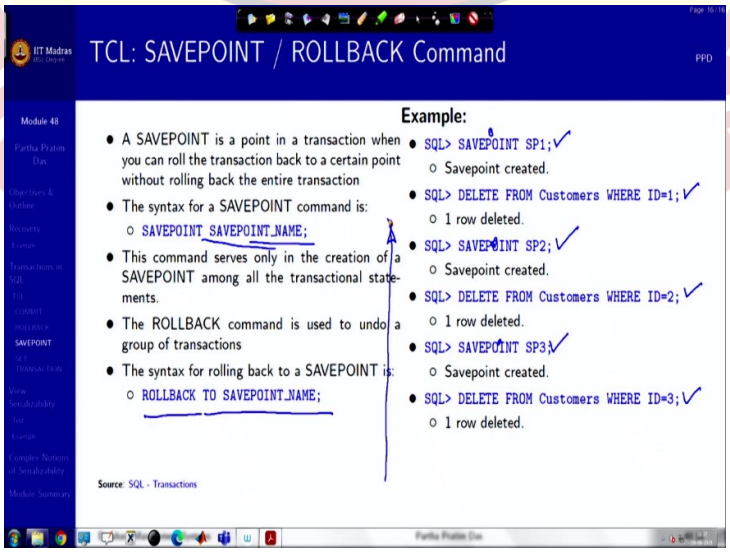




So, examples of commit command, say you have a table here, and you say where age is 25 an SQL command you are trying to do. So, age is 25 is here, it is here, so you delete, and after delete you commit, means that, changes become permanent. So, after delete what has happened? This row ID 2, record ID 2 and record ID 4 are now removed from the table, so the changes have been made permanent in the database.

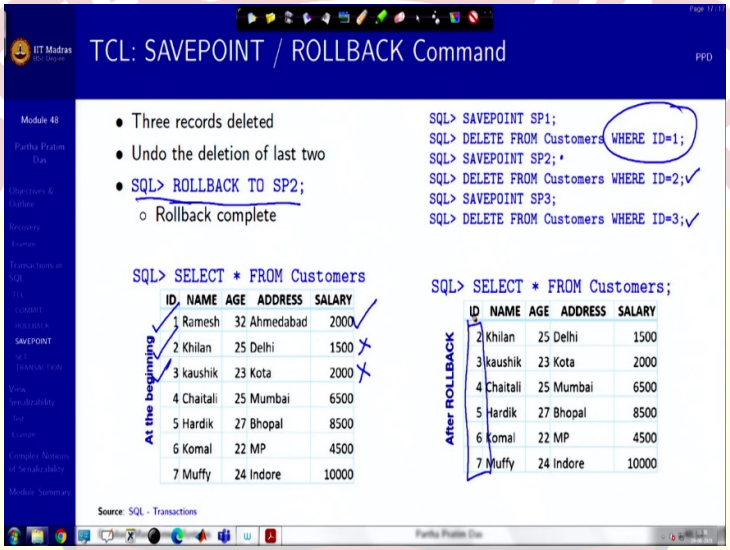
Now, if you ROLLBACK, for example, you have done this with commit, and then you have done said ROLLBACK then what will happen 2 and 4, which were deleted, they will be rolled back, that their deletions is ROLLBACK. So, after this rolling back, they are still visible in the table. So, that is a basic concept of rollback.

(Refer Slide Time: 12:25)



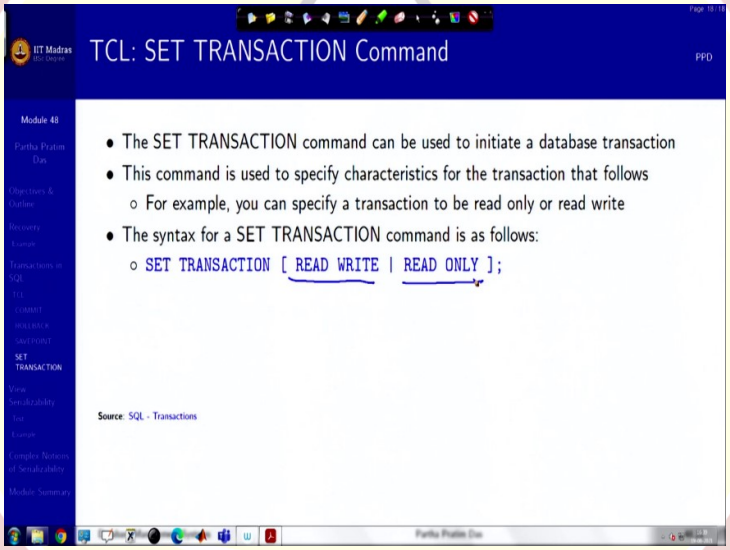
SAVEPOINT is, as I said, is you say SAVEPOINT and give it a name. And then in a ROLLBACK you can say, instead of just saying ROLLBACK, you say ROLLBACK TO that SAVEPOINT name. So, for example, you say SAVEPOINT SP1, then you have deleted ID 1. So, this is one transaction going, so we are doing one after the other. Then you SAVEPOINT the next, delete another, SAVEPOINT to the next, delete another, now you can roll back to any one of these. And if you roll back to any one of these, by roll back to that, then going backwards up to that point all changes made will be undone.

(Refer Slide Time: 13:17)



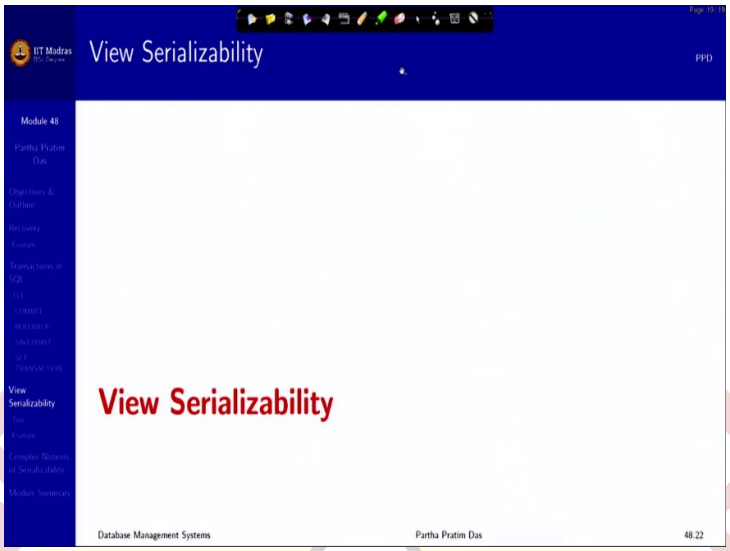
So, if we say, ROLLBACK to SP2, ROLLBACK to SP2 then this will be undone and this will be undone, but this one will remain invoke because you are not rolling back to SP1. So, this was deleted, this was deleted, this was deleted. You have ROLLBACK to undo the delete of 3, ID3, and delete of ID2, so what will remain is, only the delete of ID1. So, you can see that record ID1 is missing, but others are in place. So, in this way you can roll back to any critical point up to which you may need to roll back.

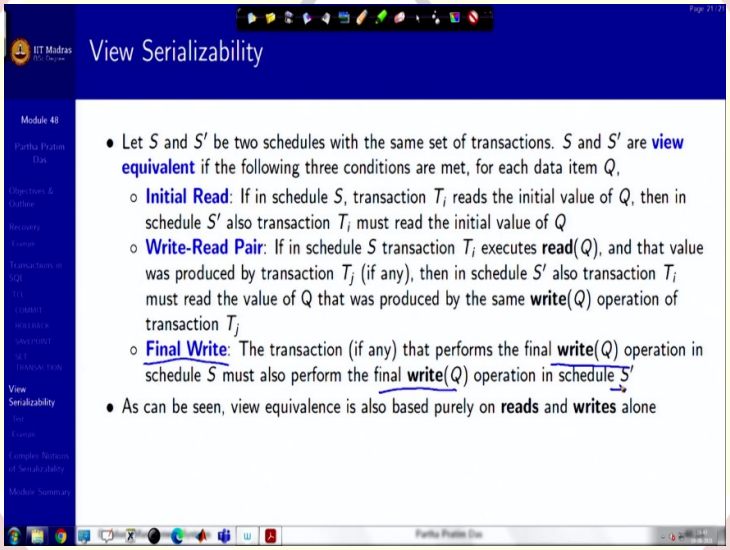
(Refer Slide Time: 14:03)



And the SAVEPOINT names, if you want to clear them out, because if I have created them, so then you can do release SAVEPOINT, SAVEPOINT name that will release the SAVEPOINT name. You can also do SET TRANSACTION to initiate a database transaction. You can say specify the type of characteristics. For example, you can say if it is a READ ONLY transaction or it is a READ WRITE transaction and so on. So, that is about the TCL.

(Refer Slide Time: 14:39)





So, now let us get into View Serializablity. We have talked about the conflict serializability, we have defined when two instructions coming from two different transactions are considered to conflict? If there is read write, write read, read write, write conflicts that are possible, and we can compute that in a given schedule by constructing the precedence graph. If precedence graph does not have a cycle, then it is conflict equivalent.

It can be changed, it can be modified by swapping, repeatedly swapping consecutive statements to convert into a serial schedule. And several schedules are not conflict serializable. A weaker notion than that is view serializability. View serializability have three conditions. So, let us look at. We have two transaction two schedules S and S prime. So, what we have, like we defined conflict equivalents by saying that one schedule is conflict

equivalent to the other, if non-conflicting transactions, consecutive transactions can be swapped to get from one schedule to the other.

Here these are similar concept, but the conditions are different. It needs to meet three conditions one is initial read. Between S and S prime if a transaction Ti reads the initial value of a data item Q then in S prime also Ti must read the initial value of Q. That is, between two schedules the initial value of a data item must be read by the same transaction. This is called Initial Read condition.

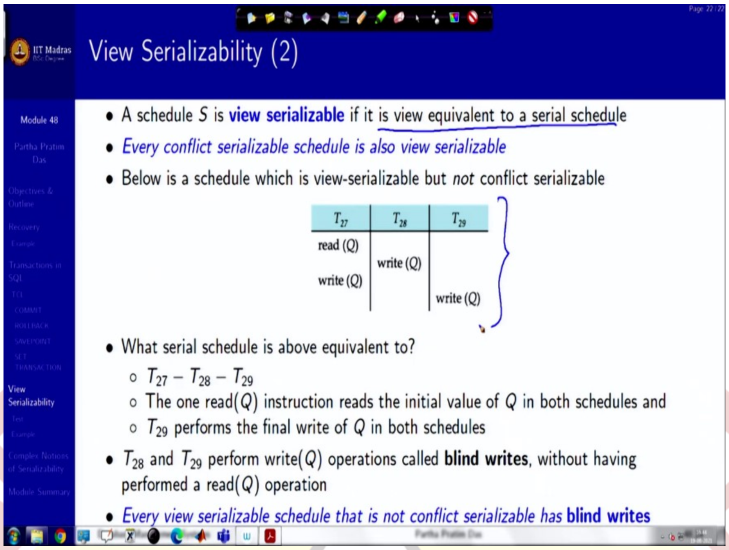
Second is Read-Write Pair. Even schedule S, transaction Ti executes read Q, it is reading Q, which is produced by transaction, some transaction Tj then in S prime also the transaction Tj, Ti must read that Q produced by the same write Q operation. So, do not get confused in Ti, Tj, it is a very simple thing.

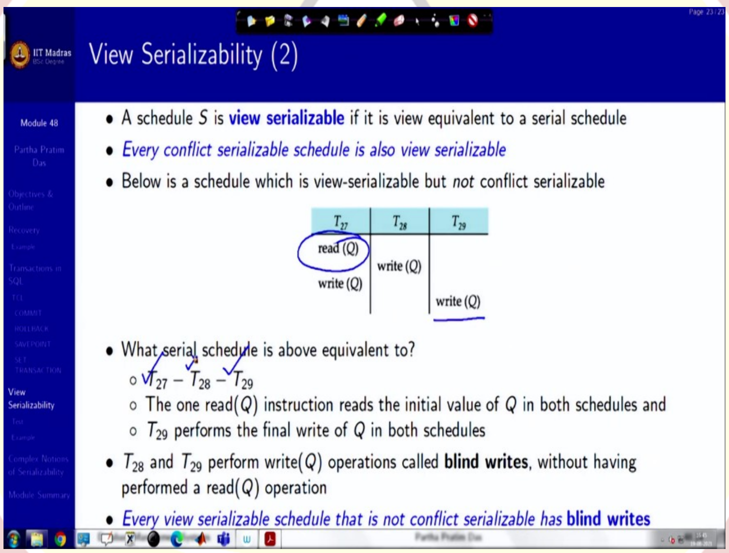
Tj wrote a variable by write Q. Naturally it did that in S, it did that in S primed. If in S, Ti read that value written by write Q in Tj in S primed also, Ti must read the value Q from that write, that is, in between there cannot be any other write. If that happens, everywhere for every variable, we say the read-write pair condition is satisfied.

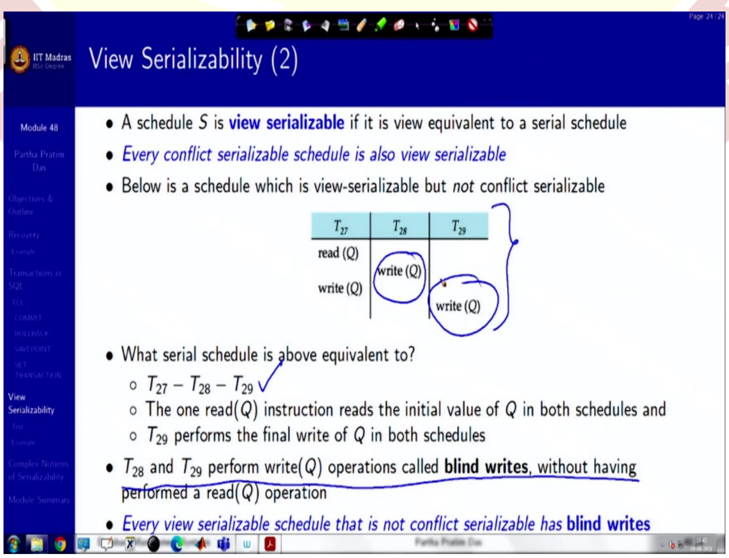
The third is called Final Write, that is, the transaction which performs the last write. Final write means, it is write-write, I mean, on the same data item there may be multiple writes. The transaction that performs a final write operation in schedule S, must also perform the final write operation in schedule S primed.

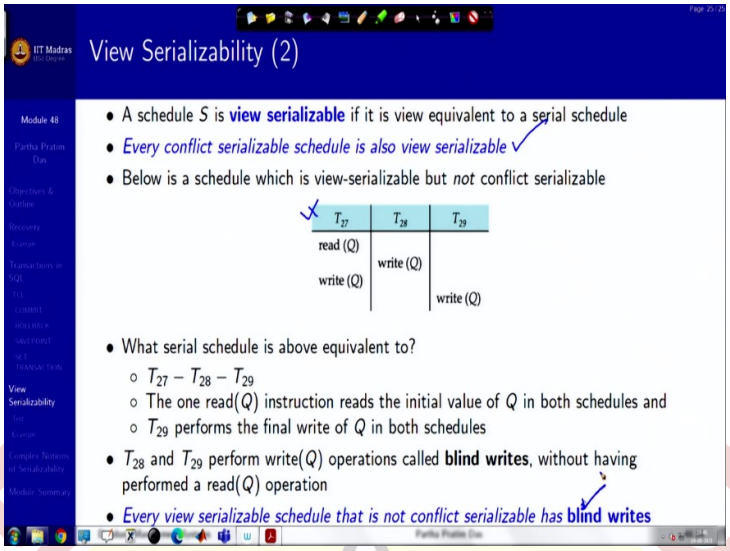
So, initial read has to match, write for the every read has to match and final writes have to match. If these three conditions match, then two schedules S and S prime are called view equivalent. And as you can see, like conflict serializability or conflict equivalence view equivalence is also purely in terms of read-writes, we do not consider the other computations in between.

(Refer Slide Time: 18:53)









So, you can see that a schedule is view serializable extending the concept of conflict serializable. A view schedule is view serializable, if it is view equivalent to a serial schedule. Now, we have seen this example before in conflict serializability, and we know, that this is not conflict serializable because it is not possible to make. I mean, the graph has cycles, and therefore, you cannot actually do any swap, and therefore, T7 and T8 cannot be serialized.

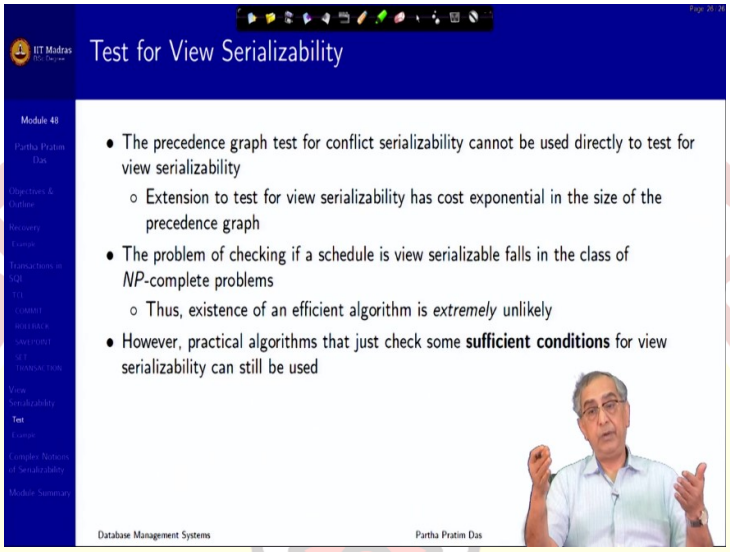
So, this is not conflict serializable, but what in terms of view serializability? There is only one read, T27. So, any schedule that starts with T27 matches the initial read condition, so T27 has to be the first. Now, the final write to Q is done by T29. So, any schedule that ends with T29 the final write would match the final write condition T29. So, if T27 is initial T29 is final, what remains is T28, which has to be in the middle.

Now, if you think about the read-write condition, write-read pair, you see that the value written has never read. So, the second condition has no premises here. So, this clearly is a serial schedule, which will be matched by this concurrent schedule, though it is not conflict serializable, but it is view serializable and will match this serial schedule in every, I mean, whatever the data is, it is easy to think in terms of.

So, this happens because, T28 and T29, if you look at this, they are doing a blind write. They just write in Q, without having read Q, so this is called a blind write. So, every view serializable schedule that is not conflict serializable like this one. This is conflict serialize, this is not conflict serializable, but this is view serializable. Any schedule which has this property has blind write. And this is something which we do not prove, but this is a result available.

So, any view serializable schedule. So, the possibility is any schedule which is conflict serializable is necessarily view serializable. Because, view serializability is a weaker condition. And any schedule, which is not conflict serializable yet view serializable must have blind writes.

(Refer Slide Time: 22:12)

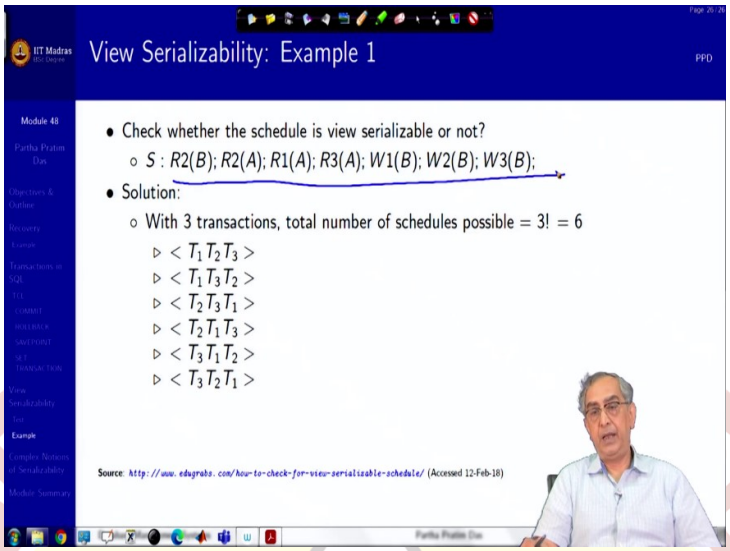


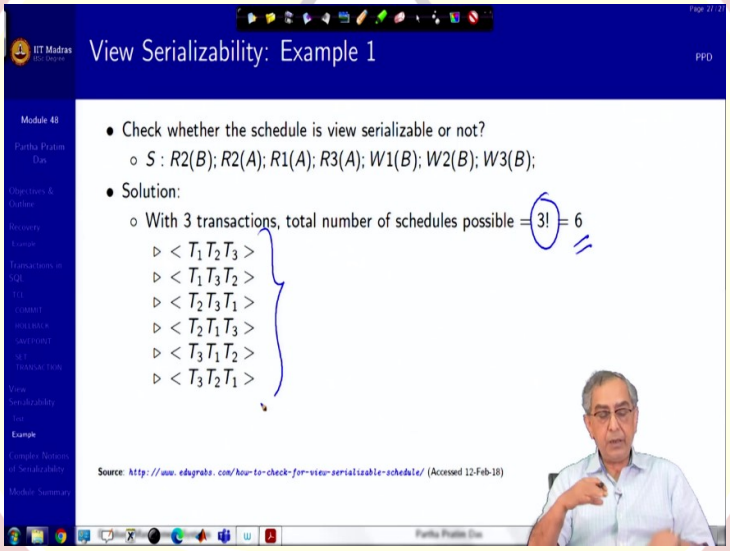
So, if we use view serializability then it will be more schedules will be possible concurrent schedule will be possible to view serial. So, like the conflict serializability we want to test for view serializability. Unfortunately, that extension of precedence graph in terms of polygraph etc, results in, has resulted in the proof that view serializability testing for view serializability is an NP-complete problem.

So, which means that, having an efficient algorithm for view serializability is extremely unlikely. So, unlike conflict serializability which we could quickly check on the precedence graph by building that graph and then doing a topological sort is not something that we can do here.

So, what he can do, we just check for some sufficient conditions, and thereby, we can find the view serializability. But it is possible that since we are checking for sufficient condition, not necessary conditions and some schedules are actually view serializable but we will not be able to detect that they are view serializable.

(Refer Slide Time: 23:30)



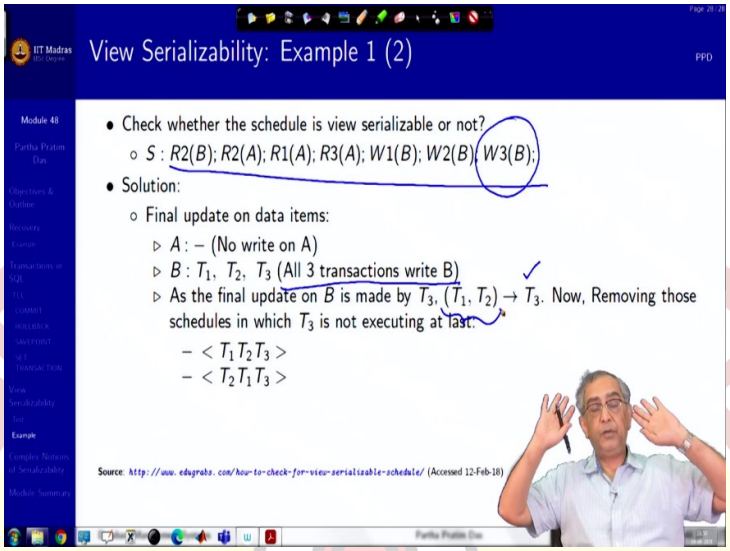


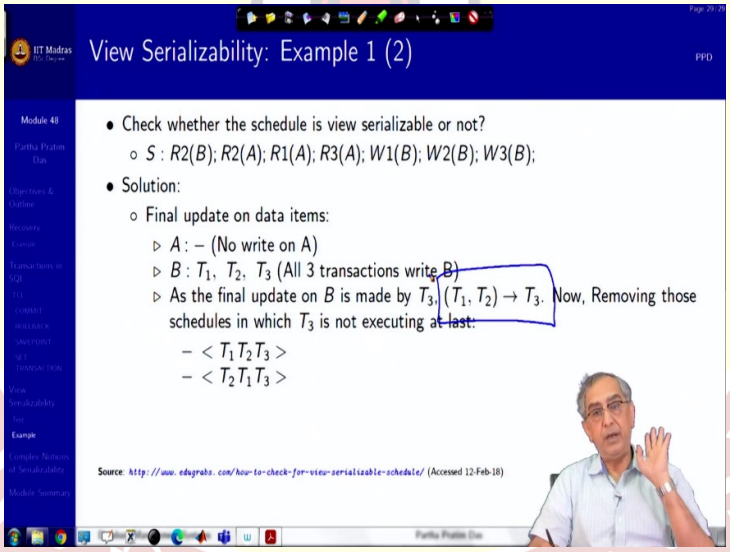
So, here is an example let us take. So, here is a schedule. Just read it carefully, here the suffixes mean, the transaction numbers. So, transaction 2 reads B, transaction 2 reads A, transaction 1 reads A, transaction 3 reads A. Similarly, transaction 1 writes B, transaction 2 writes B, transaction 3 writes B.

Now, the question is, we want to see that what are the possible schedules. What are the possible serial schedules? There are three transactions, so the number of possible serial schedules are the way you can permute these three transactions which is naturally factorial 3 that is 6, and these are the possible serial transactions, serial schedules. So, let us try to take this schedule and apply the conditions of view serializability, and check, which, I mean,

rather keep on eliminating that which are the serial schedules which cannot be equivalent to this. So, these are our 6 candidates we start with.

(Refer Slide Time: 24:54)





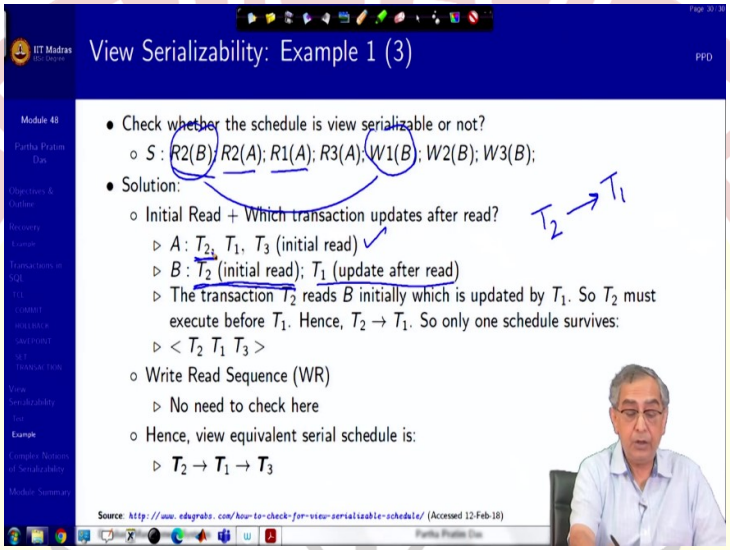
So, this is my schedule. Now, I start with the last condition, final update, just because it shrinks the data faster. Now, we can see that, there is no write to A, nobody has written A, so the final update condition on A is not is vacuously true there is no write. In terms of B, all to three transactions have written B, but the final one is by transaction 3, W3 B, which means, that transaction T1 and transaction T2 must happen in a serial schedule before T3.

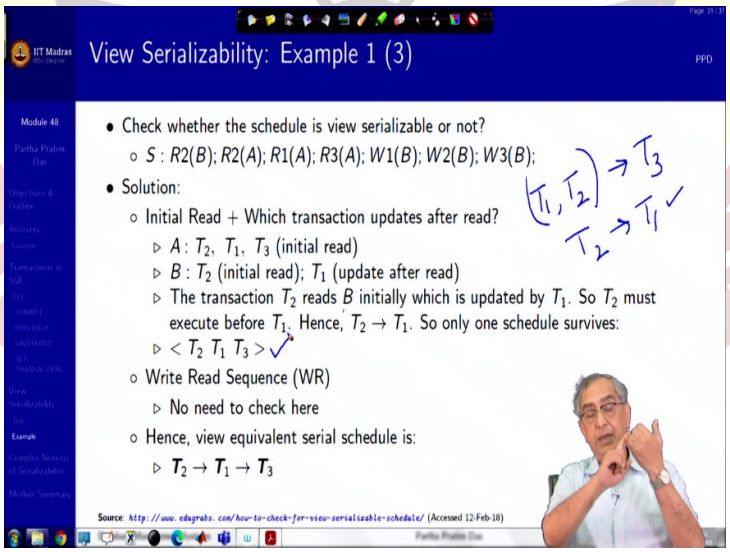
Because the condition is the final update, final write of a of the data. Here B must be done by the same transaction. So, this will have to be done by T3. So, which means that T1, now

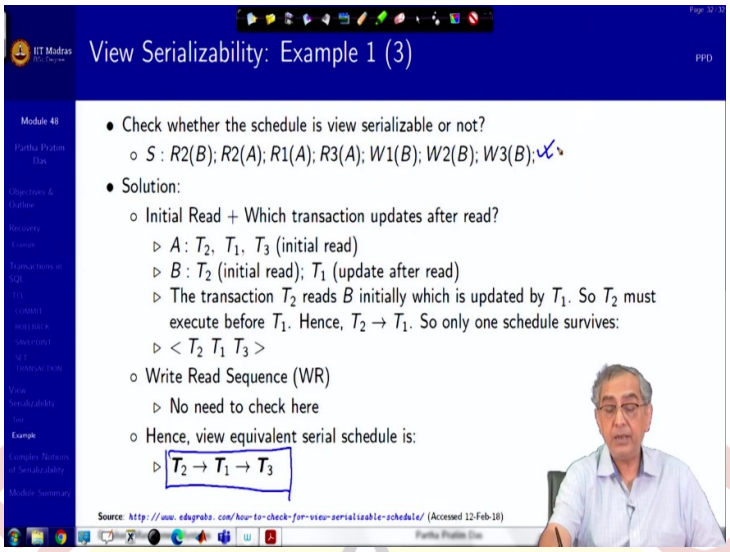
between T1, T2 two what will happen we do not know, but both of them must happen in the serial schedule before T3.

So, we express that by this you know dependency like somewhat like the precedence graph. And therefore, out of the 6, all those which does not have transaction 3 as a last transaction is eliminated, they are out, you are left with two serial schedules only. So, application of one condition we have got reduced to two schedules.

(Refer Slide Time: 26:29)







Now, let us look at initial read, right. If I look at an initial read, I have A is read by all three, B is read by first by T2 here and later updated by T1. So, in my transaction also that has to happen. So, T2 has to read what T1 writes. So, which means, T2 has to happen before T1 in a serial schedule. Similarly, for A T2 is the first read of A, and after that T1 reads, the first.

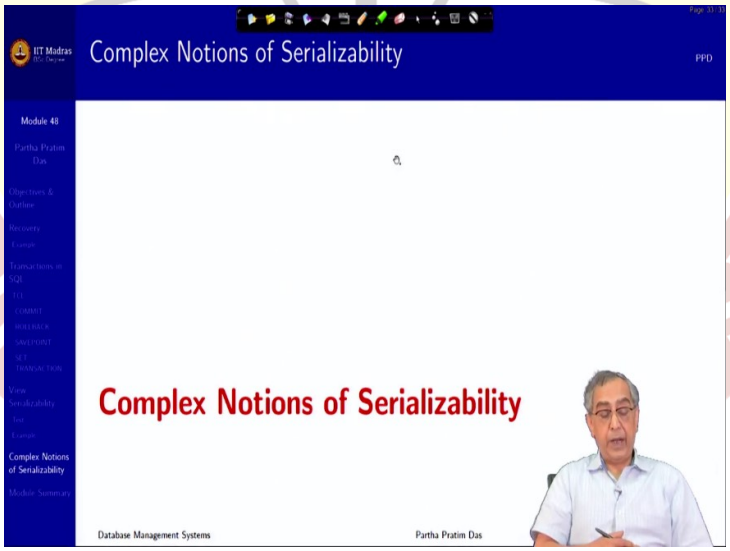
So, this will also satisfy that T2 reads A before T1 does, so we have a new dependency. Earlier what did he have we had T1, T2 has to happen before T3, now we have T2 has to happen before T1. So, out of the two, T1, T2, T3 and T2, T1, T3, the two surviving schedules, T1, T2, T3 is ruled out because it violates that, so I am left with this.

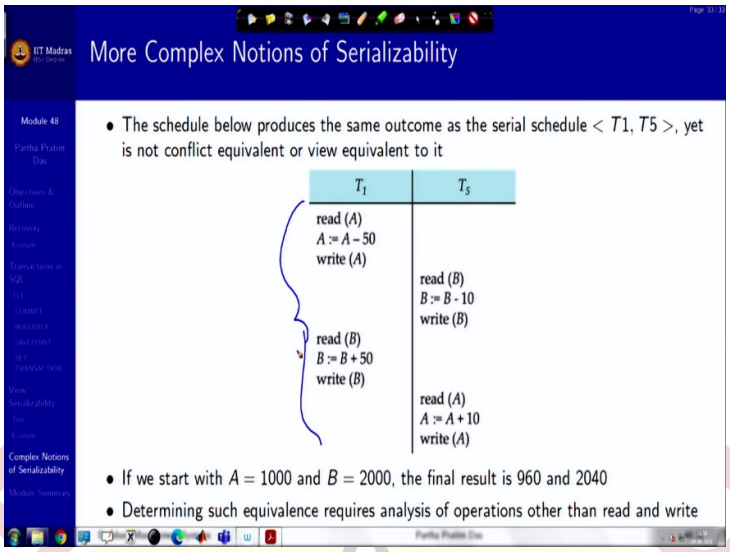
There is no write-read sequence further to check. So, we have satisfied the final write condition, initial read condition and the write-read pair condition, so this survives. So, my final serial schedule to which this particular schedule is equivalent is T2, T1, T3, therefore, this schedule is serializable. So, that is that is how you serializability works.

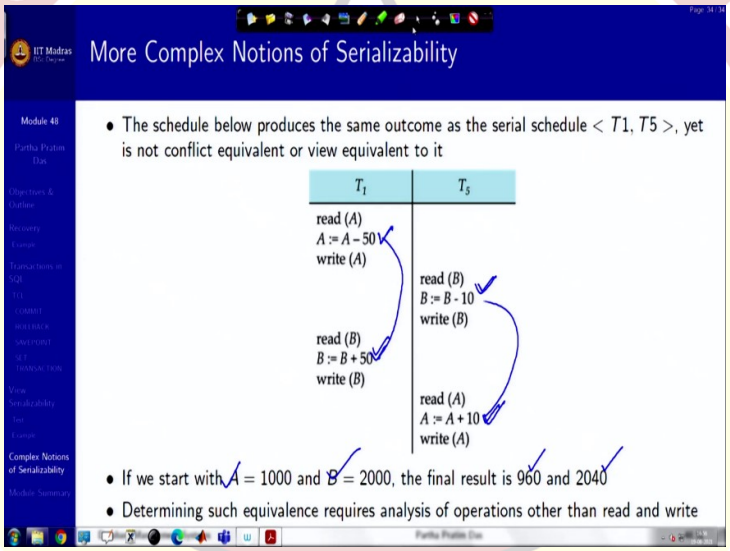
(Refer Slide Time: 28:56)

So, there is another example here little few more read-writes. And in the next, in the two slides that follow this the solution is worked out, but certainly, we have not given included them in the initial presentation, so that you can try, try to work this out, when you cannot, you look at the solutions then.

(Refer Slide Time: 29:19)







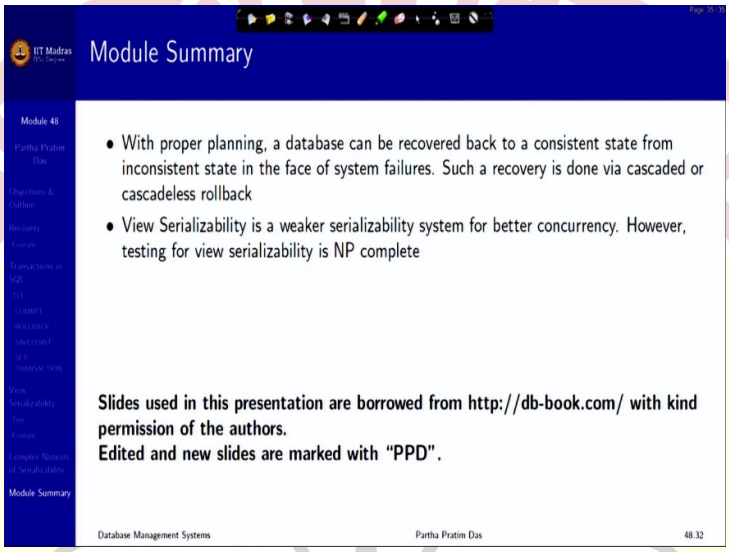
Finally, before I close, I must say that, well, that is not all about serializability because there are certain schedules which are not view serializable. If it is not conflict serializable, it can still be view serializable. If it is not even view serializable then we do not have anything else remaining.

So, if you consider this particular example, this is not view serializable. This is not, because if you look at this in this way, you will see that neither T1 followed by T5, nor T5 followed by T1 will be viewed equivalent to this. Now, but if you start with say 1000 in A and 2000 in B, this will become 950, and this will become 960, so it will be 960.

And if you start with B which is 2000, it is 1990 + 50, this will become 1040. If you do them in serial order, actually, you will also get the same thing. So, they are not view or conflict serializable, but they are serializable. So, determining such equivalence also need to consider

the other operations because just by read-write we could not have observed that this is serializable. We had to consider the actual operations being done through which we could say that it is serializable because these values match, these values match and so on. So, this is more complex, so we will not get into them. I just wanted to mention to you that that some schedules may practically be serializable because of other reasons, but they will not get detected by the way we are doing things.

(Refer Slide Time: 31:36)



So, that brings us to the end of this module. Thank you for your attention and we will continue in the next module.